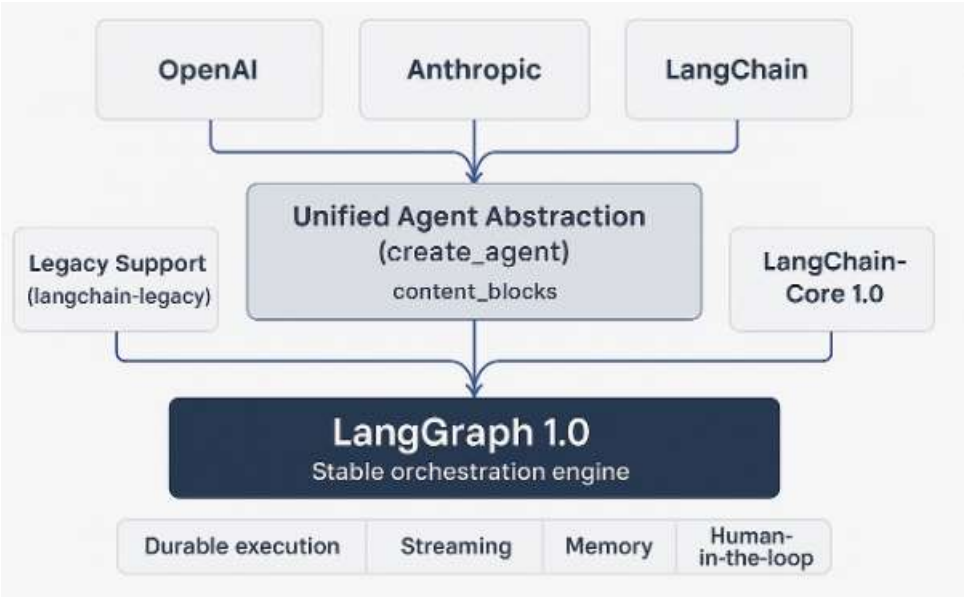
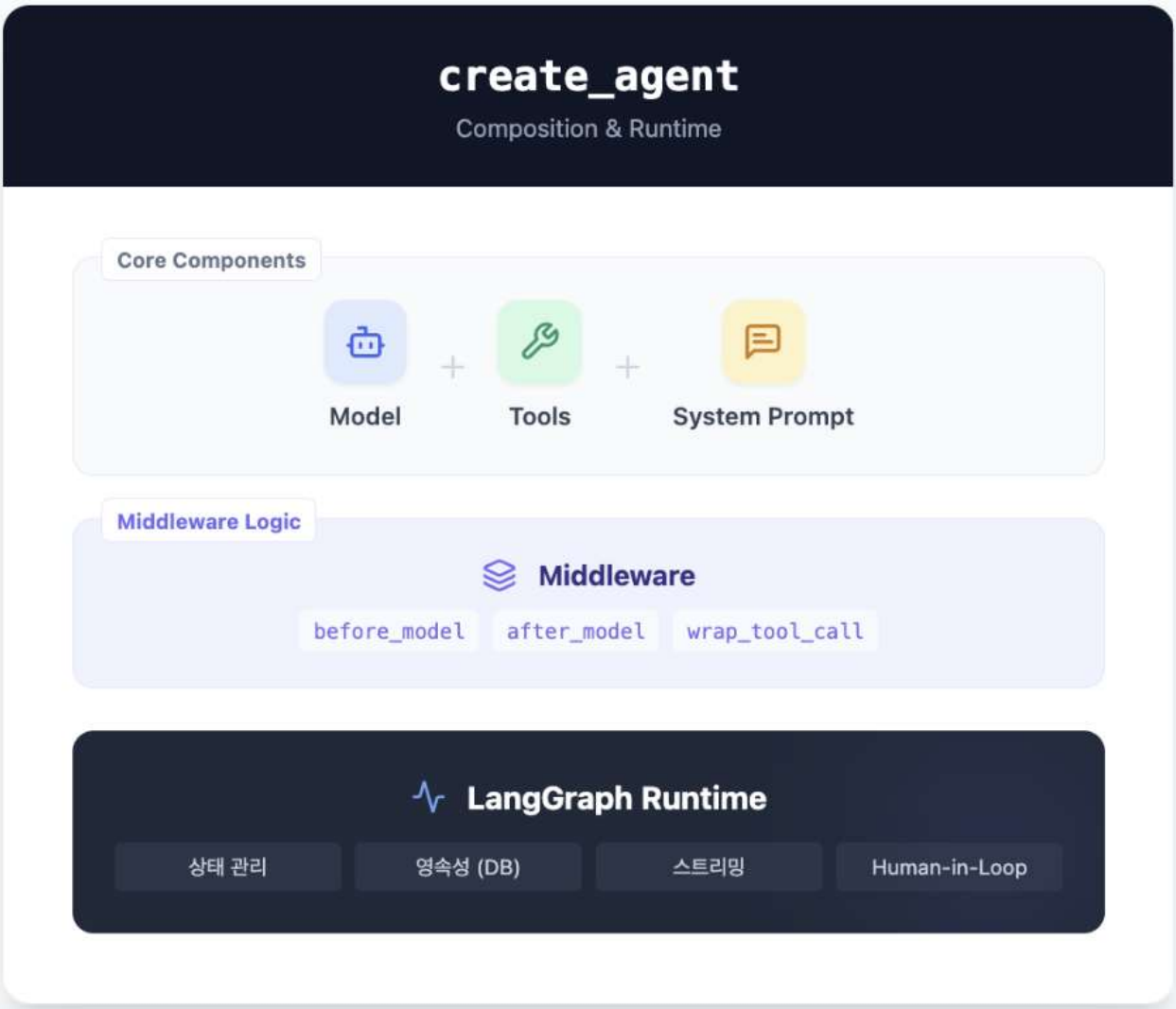


Agent

LangChain Agent

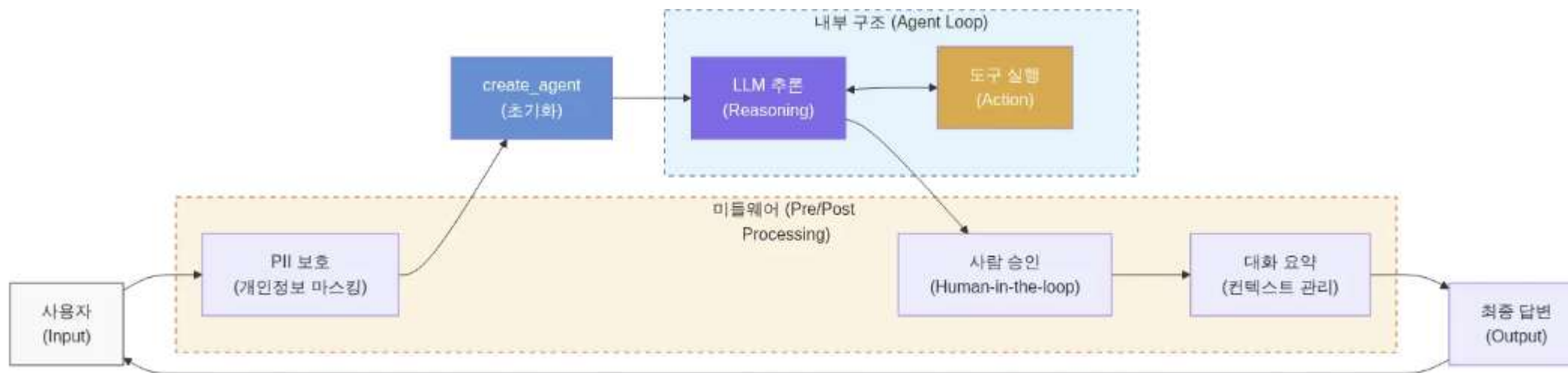
■ agent 중심 아키텍처



LangChain Agent

■ create_agent

- LangChain v1이 제공하는 고수준 에이전트 생성 API
- ReAct 패턴(추론 → 행동 → 관찰)을 내부적으로 구현
- create_agent는 내부적으로 LangGraph의 StateGraph 위에서 동작합니다



langgraph.prebuilt.create_react_agent는 LangGraph v1에서 deprecated되었습니다.
langchain.agents.create_agent 가 미들웨어, 구조화된 출력 등 더 풍부한 기능을 제공합니다.

LangChain Agent

■ create_agent의 핵심 파라미터

파라미터	설명
model	사용할 LLM 모델 (init_chat_model 또는 ChatOpenAI 등)
tools	에이전트가 사용할 도구 리스트
system_prompt	시스템 프롬프트 (에이전트의 역할 정의)
middleware	실행 흐름을 제어하는 미들웨어 리스트
checkpointer	상태 연속성을 위한 체크포인트
response_format	구조화된 출력 형식 (ToolStrategy, ProviderStrategy)

LangChain Agent

■ tool

- LLM이 스스로 판단해서 호출할 수 있는 "외부 함수"
- LLM(대형 언어 모델)이 외부 세계와 상호작용할 수 있게 해주는 핵심 컴포넌트
- 검색, 계산, 데이터베이스 조회, API 호출 등을 수행할 수 있게 만들어줍니다.
- LLM은 도구의 이름과 docstring을 보고 어떤 도구를 호출할지 결정합니다. 도구 설명이 구체적일수록 LLM이 더 정확한 판단을 합니다.
- `return_direct=True` 옵션은 Tool 결과를 LLM이 응답으로 생성하지 않고, 사용자에게 반환
- 함수를 `async def`로 정의하면 비동기 Tool로 정의됩니다.
- `model.bind_tools([tool1, tool2])`로 LLM에게 Tool을 연결해줘야 선택하고 호출 할 수 있습니다.
- `create_agent(model=..., tools=[...])`는 tool의 binding과 실행 루프를 자동으로 처리합니다
- Tool 실행 메서드 : `.invoke(args_dict)` / `.ainvoke(args_dict)`

LangChain Agent

■ @tool 데코레이터

- @tool 데코레이터는 일반 함수를 LangChain 도구로 변환합니다.
- 함수의 docstring이 도구 설명이 되어 LLM이 언제 이 도구를 사용할지 판단하는 기준이 됩니다.

```
from langchain_core.tools import tool
@tool
def calculator(expression: str) -> str:
    """
    수학 계산을 수행합니다. 사칙연산, 거듭제곱, 괄호 등을 지원합니다.
    Args:  expression: 계산할 수학 표현식 (예: "15 * 32", "(100 + 50) / 3")
    """

    import re
    # 안전한 문자만 허용 (숫자, 연산자, 괄호, 공백, 소수점)
    if not re.match(r'^[0-9+\w-*/().%Ws]+$', expression):
        return "오류: 허용되지 않는 문자가 포함되어 있습니다."
    try:
        # __builtins__를 비워 eval 보안 강화
        result = eval(expression, {"__builtins__": {}}, {})
        return f"{expression} = {result}"
    except ZeroDivisionError:
        return "오류: 0으로 나눌 수 없습니다."
    except Exception as e:
        return f"계산 오류: {str(e)}"
```

LangChain Agent

■ tool

```
from langchain_core.tools import Tool

def search(query: str) -> str:
    # 실제 검색 로직
    return f"검색 결과: {query} 관련 정보..."

tool = Tool(
    name="search",
    description="유용한 정보를 검색할 때 사용",
    func=search,
    args_schema=... # 선택
)
```

LangChain Agent

■ Built in Tool

용도	Tool
검색 도구	Tavily, Google Serper, DuckDuckGo, Bing 등
수학/계산	PythonREPLTool, LLMMathChain
문서 관련	Wikipedia, Arxiv, YouTube
파일 시스템	파일 읽기/쓰기
API 호출	REST API, GraphQL
데이터베이스	SQL, Vector Store Retriever

LangChain Agent

■ Tool의 고급 기능

- Tool Calling (Function Calling): GPT-4o, Claude 3 등 최신 모델이 네이티브로 지원
- Structured Output: Pydantic으로 강력한 입력/출력 검증
- Custom Tool with BaseTool 상속
- Async Tool (async def)
- Human-in-the-loop Tool (승인 필요)
- Toolkits: 여러 도구를 묶은 세트 (예: SQLDatabaseToolkit)

LangChain Agent

■ init_chat_model

- init_chat_model은 프로바이더:모델명 형식으로 다양한 LLM을 통일된 인터페이스로 초기화합니다

```
from langchain.chat_models import init_chat_model

# OpenAI 모델
model = init_chat_model("openai:gpt-4.1-mini", temperature=0)

# Anthropic 모델 (langchain-anthropic 설치 필요)
# model = init_chat_model("anthropic:claude-sonnet-4-5-20250929", temperature=0)

# Google 모델 (langchain-google-genai 설치 필요)
# model = init_chat_model("google_genai:gemini-2.0-flash", temperature=0)
```

LangChain Agent

■ 기본 에이전트 예제

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.tools import tool
# 도구 정의
```

@tool

```
def search_web(query: str) -> str:
    """웹에서 정보를 검색합니다."""
    return f"'{query}'에 대한 검색 결과입니다."
```

@tool

```
def calculate(expression: str) -> str:
    """수학 계산을 수행합니다."""
    return str(eval(expression))
```

에이전트 생성

```
agent = create_agent(
    model=init_chat_model("gpt-4o-mini"),
    tools=[search_web, calculate],
    system_prompt="당신은 웹 검색과 계산을 도와주는 AI 어시스턴트입니다.",
)
```

에이전트 실행

```
result = agent.invoke({
    "messages": [{"role": "user", "content": "2024년 한국 GDP는 얼마야?"}]
})
```

LangChain Middleware

■ Middleware

- middleware는 에이전트의 동작을 가로채서 변환하는 계층입니다.
- 에이전트 코드를 수정하지 않고도 PII 보호, 대화 요약, 사람 승인 등의 기능을 추가할 수 있습니다.



Middleware가 해결하는 문제 :

긴 대화로 인한 Context Window 초과

민감한 Tool 호출 시 인간 승인 필요 (Human-in-the-Loop)

비용 폭증, 무한 루프 방지

PII(개인정보) 보호, Guardrails 적용

Retry, Fallback, Logging, Prompt 변환 등

LangChain Middleware

■ Middleware

- AgentMiddleware를 상속받은 클래스는 아래 메서드들을 선택적으로 오버라이드합니다

Hook	실행시점	용도
before_agent	Agent 실행 시작 전(1회)	초기화, 세션 검증
before_model	모델 호출 직전 (매 턴마다)	상태 확인, 조기 종료(jump_to)
wrap_model_call	모델 호출을 감싸서	재시도, 모델 교체, 요청 변형
after_model	모델 응답 직후	응답 검증, 사람 승인 요청
wrap_tool_callTool	Tool 호출을 감싸서	재시도, 모니터링, 결과 변형
after_agentAgent	Agent 실행 종료 후 (1회)	정리, 최종 로깅

LangChain Middleware

■ Middleware

```
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware

class LoggingMiddleware(AgentMiddleware):
    def before_model(self, state, runtime=None):
        print(f"[before_model] 현재 메시지 수: {len(state['messages'])}")

    def after_model(self, state, runtime=None):
        last = state["messages"][-1]
        print(f"[after_model] 모델 응답: {last.content!r}")

agent = create_agent(model=model, tools=[add], middleware=[LoggingMiddleware()])
agent.invoke({"messages": [HumanMessage(content="2+3은?")]}))
```

```
class ToolLoggingMiddleware(AgentMiddleware):
    def wrap_tool_call(self, request, handler):
        print(f"[wrap_tool_call] 호출 전: {request.tool_call['name']}({request.tool_call['args']})")
        result = handler(request) # 실제 Tool 실행. 여러 번 부르면 재시도 로직이 됨
        print(f"[wrap_tool_call] 호출 후: {result.content!r}")
        return result
```

LangChain Middleware

■ Middleware

middleware	실행시점
HumanInTheLoopMiddleware	"위험한 Tool 실행 전 사람 승인" 패턴을 표준 Middleware로 제공
SummarizationMiddleware	대화가 길어져 토큰 한도에 가까워지면 자동으로 이전 대화를 요약
LLMToolSelectorMiddleware	Tool이 너무 많을 때 LLM으로 관련 있는 것만 먼저 추려서 메인 모델에 전달
ModelFallbackMiddleware	모델 호출 실패 시 대체 모델로 자동 전환
PIIMiddleware	대화에서 개인정보 탐지/마스킹
ToolRetryMiddleware	실패한 Tool 호출을 백오프하며 자동 재시도
ToolCallLimitMiddleware	모델 호출 횟수 제한, Tool/모델 호출 횟수에 상한을 걸어 무한 루프 방지

#여러 Middleware를 함께 쓸 때
agent = create_agent(model=model, tools=tools, middleware=[A(), B(), C()])